ANDROID SECURITY ASSESSMENT

Fahem Native

Intent-Based Authentication Bypass & Native Library Flag Disclosure

---

CTF Writeup — FahemSec Platform

Prepared by: Mohammad

Category: Android — Client-Side & Native Library Reverse Engineering

Status: Private draft — pending confirmation of challenge closure before publication

August 2026

1. Environment Setup

The Kali attacker machine was connected to the Android emulator over ADB to establish a working attack channel before any analysis began.

```
adb connect 192.168.1.11
adb devices
# List of devices attached
# 192.168.1.11:5555   device
```

Figure 1 — Establishing ADB connectivity between Kali and the emulator

The target APK was then installed. The first attempt failed with INSTALL_FAILED_TEST_ONLY, since the package was built as a test-only artifact; adding the -t flag permitted the install.

```
adb install -t fahemnative.apk
# Performing Streamed Install
# Success
```

Figure 2 — Installing the APK with the test-only override flag

2. Application Reconnaissance

With the app installed, the login/register screen was reviewed first to understand the intended user flow before looking for ways around it.

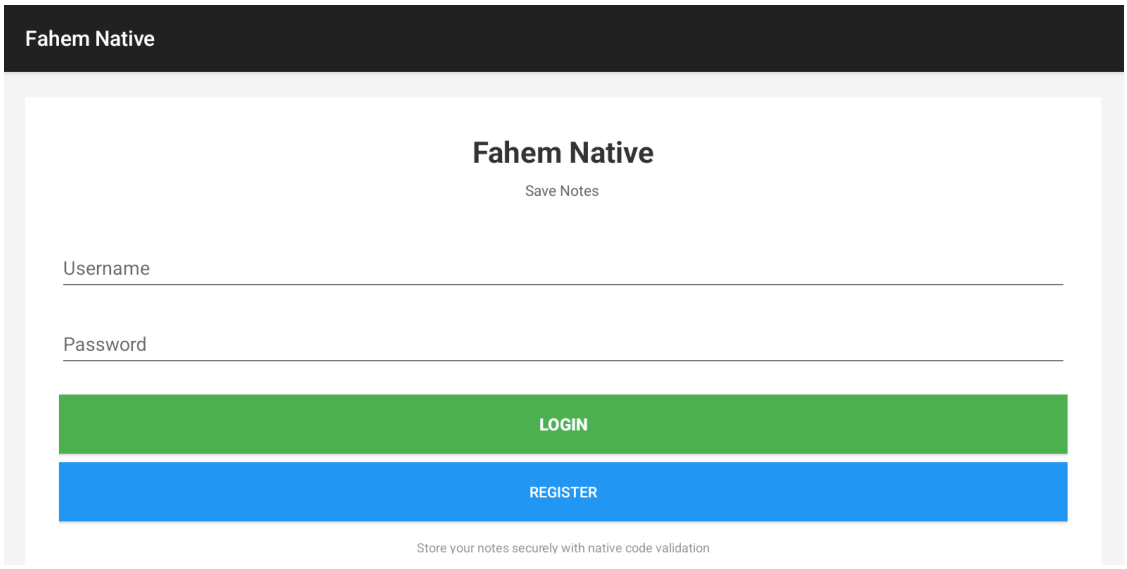

Figure 3 — Fahem Native login / register screen

After registering through the normal UI, the main screen exposes three actions: Add Note, View Notes, and Logout. A footer note on this screen is a deliberate hint from the challenge author:

> *"UI input limited to 20 characters. For longer notes, use intent: adb shell am start -a android.intent.action.VIEW -d \"fahem://add\" --es note \"Your note here\""*

This hint confirms two things worth noting explicitly: the developers were aware the UI layer restricts input length, and any note longer than that limit can only be delivered by constructing a raw Intent directly — which is a strong signal that the logic worth testing lives past the UI's own client-side validation, not within it.

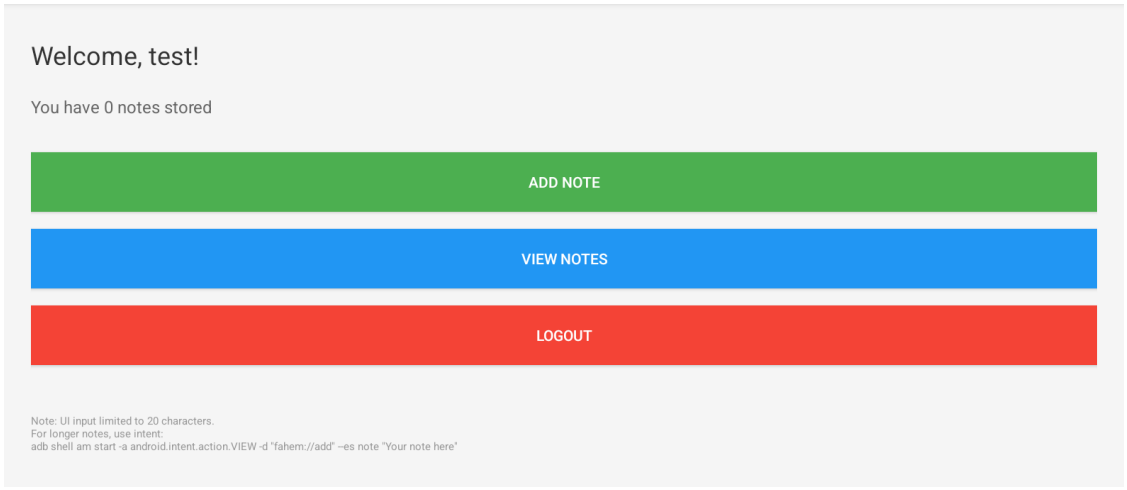

Figure 4 — Post-registration screen showing Add Note / View Notes / Logout, with the intent hint in the footer

3. Static Analysis

3.1 Recovering the package name

The APK was decoded with apktool to recover a readable manifest and identify the application's package name, which is required for every subsequent Drozer command.

```
apktool d fahemnative.apk -o fahem_decoded
cd fahem_decoded
cat AndroidManifest.xml | grep "package"
```

Resulting package name: com.ctf.timedhook

3.2 Enumerating the attack surface with Drozer

A Drozer console session was connected to the emulator to enumerate exported components — the parts of the app reachable by any other app on the device without needing to go through the intended UI.

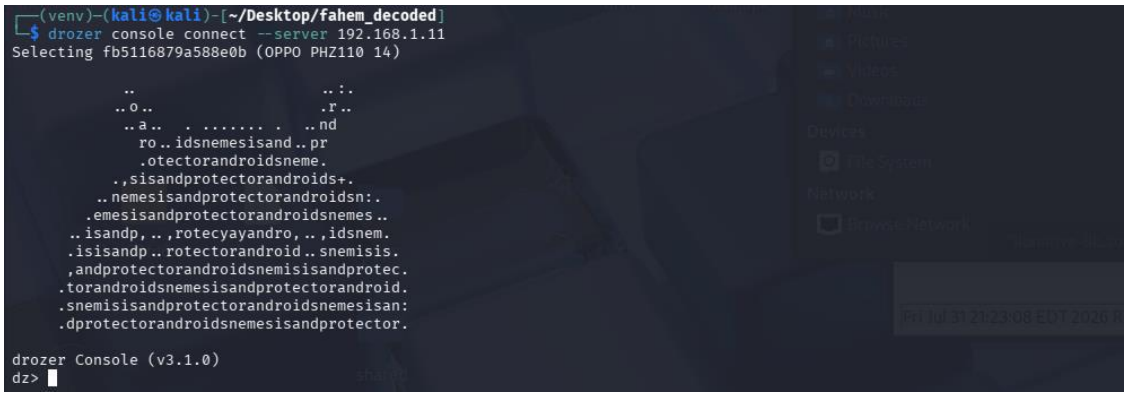

Figure 5 — Drozer console connected to the target device

```
dz> run app.package.attacksurface com.ctf.timedhook

Attack Surface:
  1 activities exported
  1 broadcast receivers exported
  0 content providers exported
  0 services exported
  is debuggable
```

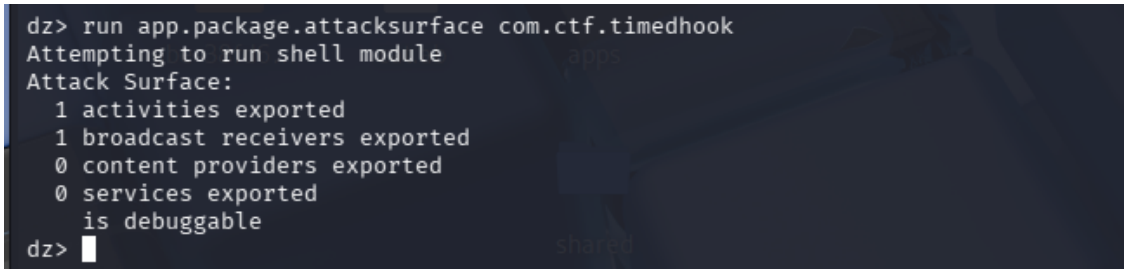

Figure 6 — Attack surface scan output

Following up with app.activity.info and app.broadcast.info narrowed this down to two exported components: MainActivity, which required no permission, and androidx.profileinstaller.ProfileInstallReceiver, a stock Jetpack component gated behind the signature-level android.permission.DUMP. The latter is not attacker-controllable and was ruled out, leaving MainActivity as the sole realistic target.

3.3 Manifest review

The manifest confirms MainActivity is exported with no permission requirement, and that it accepts input through three separate entry points: its normal launcher intent-filter, a custom fahem:// deep link, and a custom com.ctf.timedhook.ADD_NOTE action.

```
<activity
    android:name="com.ctf.timedhook.MainActivity"
    android:exported="true"
    android:launchMode="singleTask">
    <intent-filter>
        <action android:name="android.intent.action.MAIN"/>
        <category android:name="android.intent.category.LAUNCHER"/>
    </intent-filter>
    <intent-filter>
        <action android:name="android.intent.action.VIEW"/>
        <category android:name="android.intent.category.DEFAULT"/>
        <category android:name="android.intent.category.BROWSABLE"/>
        <data android:scheme="fahem"/>
    </intent-filter>
    <intent-filter>
        <action android:name="com.ctf.timedhook.ADD_NOTE"/>
        <category android:name="android.intent.category.DEFAULT"/>
    </intent-filter>
</activity>
```

3.4 Reading the source with jadx-gui

The APK was opened directly in jadx-gui (no manual extraction needed) to read the decompiled Kotlin/Java source alongside the manifest.

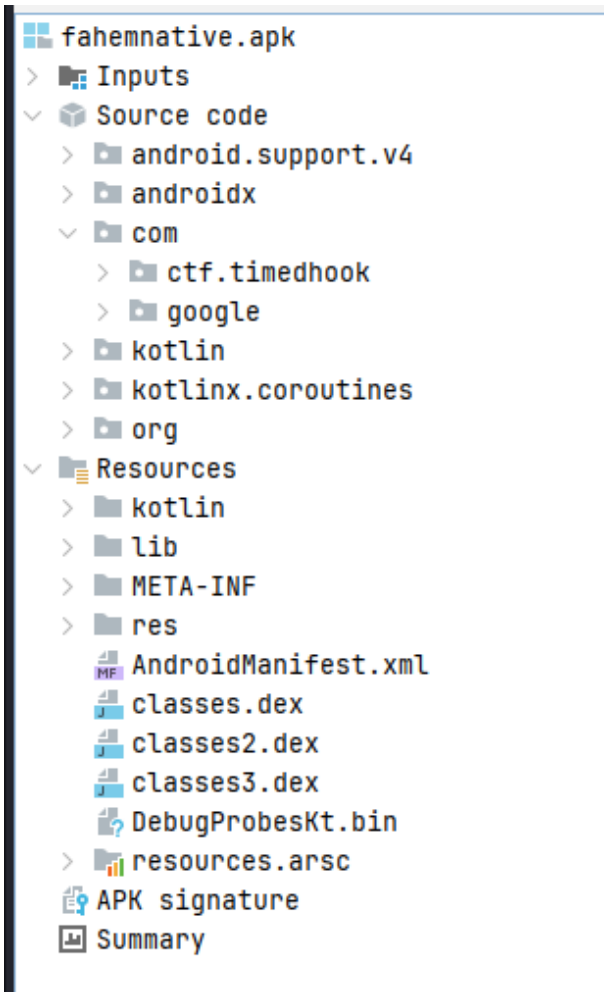

Figure 7 — jadx-gui source tree for com.ctf.timedhook

MainActivity.onCreate() shows that whenever the activity starts, it checks a stored login flag in SharedPreferences, loads any existing notes and passes them to a native checkNotes() call, then unconditionally calls handleIntent() on whatever Intent launched it — regardless of login state.

```
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    this.prefs = getSharedPreferences(PREFS_NAME, 0);
    boolean isLoggedIn = this.prefs.getBoolean(KEY_LOGGED_IN, false);
    if (isLoggedIn) {
        ArrayList<String> notesList = loadNotes();
        if (notesList.size() > 0) {
            String[] notesArray = (String[]) notesList.toArray(new String[0]);
            checkNotes(notesArray, notesList.size());
        }
        showMainScreen();
    } else {
        showLoginScreen();
    }
    handleIntent(getIntent());
}
```

3.5 Finding #1 — Authentication bypass in handleIntent()

This is the most significant finding in the assessment. handleIntent() is reached on every single launch of MainActivity — including launches triggered entirely from outside the app, since the activity is exported. Its logic is:

```
private void handleIntent(Intent intent) {
    if (intent != null && "android.intent.action.VIEW".equals(intent.getAction())) {
        String username = intent.getStringExtra(KEY_USERNAME);
        String password = intent.getStringExtra(KEY_PASSWORD);
        String note = intent.getStringExtra("note");
        if (username != null && password != null && note != null) {
            SharedPreferences.Editor editor = this.prefs.edit();
            editor.putString(KEY_USERNAME, username);
            editor.putString(KEY_PASSWORD, password);
            editor.putBoolean(KEY_LOGGED_IN, true);
            editor.apply();
            addNote(note);
            showMainScreen();
            return;
        }
        if (note != null && this.prefs.getBoolean(KEY_LOGGED_IN, false)) {
            addNote(note);
        }
    }
}
```

The critical issue is that handleIntent() treats the presence of the username, password, and note extras as sufficient proof of a valid registration. There is no credential check, no server round-trip, and no verification that the caller is the app's own UI rather than an arbitrary external component. Any app — or, in testing, Drozer acting on the tester's behalf — can deliver a VIEW intent carrying those three extras directly to MainActivity and be granted a fully authenticated session instantly, with an attacker-chosen note stored in the same step. This is a complete bypass of the intended registration and login flow, not merely a way to skip the UI.

A secondary path in the same method is also worth flagging: once KEY_LOGGED_IN is true, any subsequent VIEW intent carrying only a note extra is accepted and stored without any further check — meaning the note-injection primitive remains available indefinitely after the first bypass, not just during account creation.

Proof of concept, executed through Drozer:

```
dz> run app.activity.start --action android.intent.action.VIEW \
    --component com.ctf.timedhook com.ctf.timedhook.MainActivity \
    --extra string username "hacker" \
    --extra string password "hacker" \
    --extra string note "test"
```

This single command creates an authenticated session and stores a note, entirely from outside the application and without touching its login screen.

4. Native Library Analysis

checkNotes() is declared as a native method backed by a bundled shared library, meaning its real logic is not visible in the decompiled Java/Kotlin source and requires binary reverse engineering to inspect.

```
public native void checkNotes(String[] strArr, int i);

static {
    System.loadLibrary("native-lib");
}
```

```
public native void checkNotes(String[] strArr, int i);

static {
    System.loadLibrary("native-lib");
}
```

Figure 8 — Native method declaration in MainActivity

libnative-lib.so was extracted from fahem_decoded/lib/ and loaded into Ghidra for decompilation. The symbol tree revealed two functions of interest beyond the JNI bridge itself: vulnerableCheckNotes and printFlag.

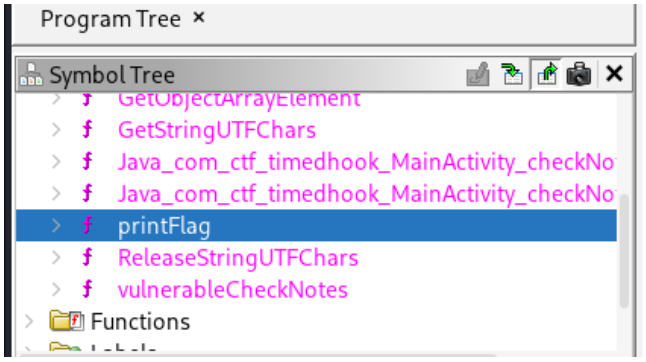

Figure 9 — Ghidra symbol tree for libnative-lib.so

4.1 vulnerableCheckNotes

This function copies each note into a 72-byte stack buffer through __strcpy_chk with an explicit 64-byte bound, and logs both the raw note and the buffer's contents via __android_log_print under the CTFNative tag. Because the copy is FORTIFY-bounded and the function is protected by a stack canary, this does not represent a genuinely exploitable overflow — its name and structure imitate a classic vulnerable strcpy pattern, but the runtime protections remain intact. In practice its only real effect is writing note content to logcat, which made it useful for observation but not for exploitation.

4.2 printFlag — Finding #2

printFlag is the function that actually gates the flag, and its logic is a length check rather than a real memory-safety flaw:

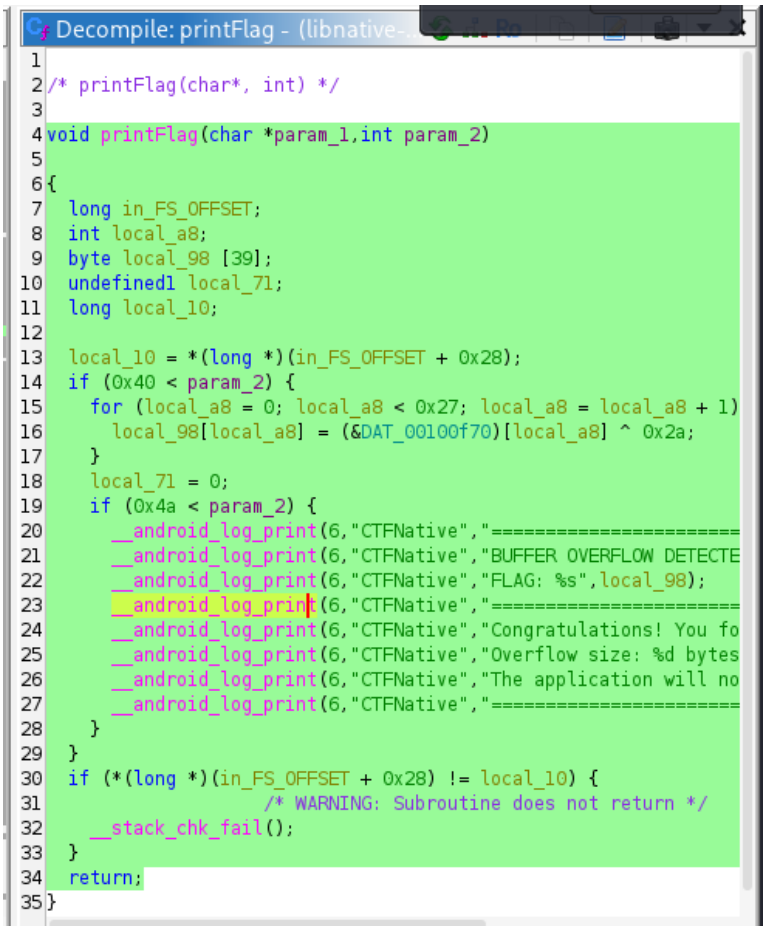

Figure 10 — Ghidra decompilation of printFlag

```
void printFlag(char *param_1, int param_2) {
    ...
    if (0x40 < param_2) {
        for (local_a8 = 0; local_a8 < 0x27; local_a8++) {
            local_98[local_a8] = (&DAT_00100f70)[local_a8] ^ 0x2a;
        }
        if (0x4a < param_2) {
            __android_log_print(6, "CTFNative", "BUFFER OVERFLOW DETECTED!");
            __android_log_print(6, "CTFNative", "FLAG: %s", local_98);
            ...
        }
    }
    ...
}
```

Once the note length (param_2) exceeds 64 characters, a hardcoded 39-byte buffer is decoded by XOR-ing it with the constant 0x2A. Once the length exceeds 74 characters, that decoded buffer is printed to logcat labeled as the flag. No actual buffer overflow occurs anywhere in this function — the length value is used purely as a conditional gate, and the "BUFFER OVERFLOW DETECTED" messaging is cosmetic, designed to walk the tester toward the length threshold rather than reflecting real memory corruption.

5. Exploitation

Combining both findings gives a direct path to the flag: the authentication bypass in handleIntent() (Finding #1) delivers attacker-controlled note content with no login required, and printFlag (Finding #2) reveals the flag once that note exceeds 74 characters. An 80-character note was sent to comfortably clear the threshold.

```
dz> run app.activity.start --action android.intent.action.VIEW \
    --component com.ctf.timedhook com.ctf.timedhook.MainActivity \
    --extra string note
"AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA"
```

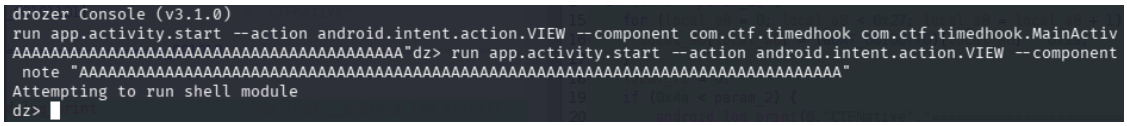

Figure 11 — Delivering an 80-character note through Drozer

logcat was monitored for the native library's log tag to capture the result:

```
adb logcat -s CTFNative

08-01 05:40:04.031  I CTFNative: --- Processing Note 1 ---
08-01 05:40:04.031  I CTFNative: Length: 80 chars
08-01 05:40:04.031  E CTFNative: WARNING! Note 1 exceeds maximum size!
08-01 05:40:04.031  E CTFNative: Buffer overflow imminent...
08-01 05:40:04.031  E CTFNative: ====================================
08-01 05:40:04.031  E CTFNative: BUFFER OVERFLOW DETECTED!
08-01 05:40:04.031  E CTFNative: FLAG: flag{Bu773r_0v3rfl0w_1n_N4t1v3_Andorid}
08-01 05:40:04.031  E CTFNative: ====================================
08-01 05:40:04.031  E CTFNative: Congratulations! You found the vulnerability!
```

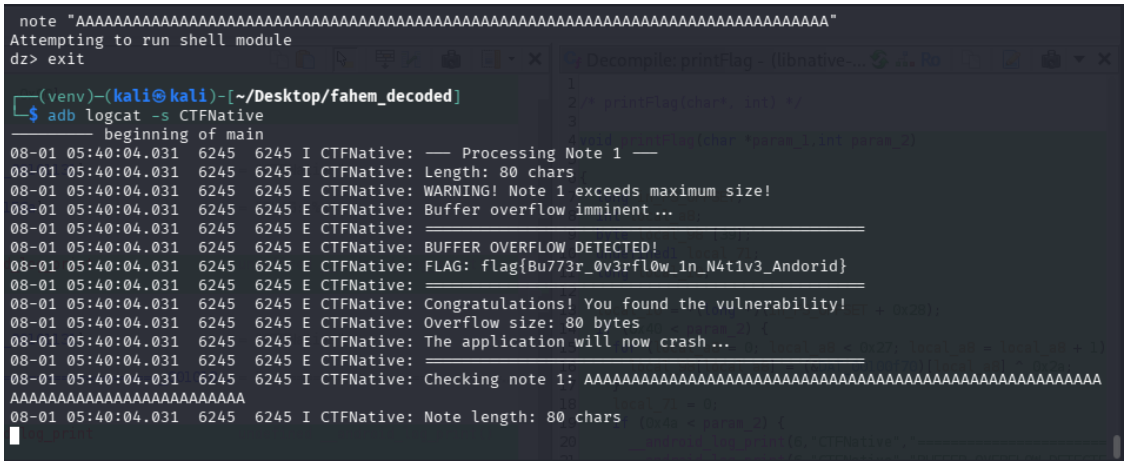

Figure 12 — logcat output showing the captured flag

Flag: `flag{Bu773r_0v3rfl0w_1n_N4t1v3_Andorid}`

6. Root Cause Summary

#	Finding	Classification
1	Exported MainActivity accepts unauthenticated username/password/note extras and silently creates a logged-in session with no validation	Improper Access Control on an Exported Component (CWE-926) — OWASP Mobile M8: Security Misconfiguration, adjacent to M3: Insecure Authentication

2	printFlag discloses a hardcoded secret based solely on client-supplied note length; no genuine memory corruption occurs	Hardcoded secret / insecure design in native code — OWASP Mobile M10: Insufficient Cryptography (reversible XOR obfuscation)

7. Remediation

- Set android:exported="false" on MainActivity, or enforce a signature-level permission, so only the application itself can deliver session-creating intents.
- Never treat Intent extras as trusted authentication input. Session creation should require a genuine credential-verification step, ideally validated server-side, rather than accepting client-supplied values at face value.
- Remove verbose logging of user-controlled input and any sensitive constants from production builds — the CTFNative tag currently logs both note content and the secret flag in cleartext.
- Do not embed secrets client-side using reversible obfuscation such as XOR. If a secret must exist in a mobile binary at all, it should be protected with proper cryptography and any "unlock" condition validated server-side, not derived from a client-supplied value.

Note: This challenge is/was active on the FahemSec platform. This document is intended for private reference and will only be published once the challenge is confirmed closed for public writeups.